

Review Questions

Capítulo 2

1._ A, D y G

Los operadores que se pueden usar con boolean son '==', '!' y el 'cast (boolean)'. Aunque el cast no es útil en la práctica, es válido porque boolean es un tipo en Java.

2._ A, B y D.

byte, short y char siempre se convierten a int en operaciones. Java lo hace en automático.

3._ B, C, D y F

Las opciones correctas funcionan porque convierten el valor a int, cambian el tipo de ear o hacen que la variable de destino sea long.

4._ B

Primero, teeth vale 20, entonces teeth != 10 es true. Luego, en (wolf = false) no está comparando, sino que está cambiando el valor de wolf a false, y además esa expresión devuelve false. Entonces queda true - false, y eso da true.

Al final, canine queda en true, teeth sigue siendo 20 y wolf termina en false.

5._ A y C

Estas son correctas porque sí siguen el orden de menor a mayor prioridad. Las demás están desordenadas o al revés.

6._ F

El código no compila porque en el método se está regresando un valor que no es del tipo correcto. El (int) solo se aplica a fruit, y luego se suma con vegetables que es float, entonces el resultado final es float. Pero el método dice que debe regresar un long, y Java no convierte automáticamente de float a long. Por eso marca error.

7._ D

clear es true, safe es false porque no se evalúa todo, y tasty es false porque se decrementa ph. Resultado: true- false- false.

8._ A

pig se queda en 4 por el post-incremento, y goat pasa de 2 a 1. Resultado: 4 - 1.

9._ A, D y E

Imprime 4, luego 5 y luego 1, por cómo funcionan los ternarios y el post-incremento.

10._G

El código no compila porque en la línea de zebra, aunque se hace cast a byte, al multiplicar Java convierte automáticamente los valores a int. Entonces el resultado es int, pero lo están guardando en un short, y eso no se puede sin cast

11._D

Este se resuelve de izquierda a derecha y con paréntesis primero: resultado 2, 0, 5.

12._D

x++ regresa el valor original y --x regresa el valor ya decrementado.

Cambiaría el número si primero viniera ++ y luego x, porque en este comando diría: “súmalo y luego incrementalo”, en caso contrario como lo que sucedió aquí dice: “muéstramelo y luego incrementa” en el X++.

13._F

En goingToTheStore, se evalúa de izquierda a derecha: sunny & raining da false, porque true & false es false. Luego false ^ sunday da true, porque ^ es como “diferente”, entonces false ^ true = true.

En goingToTheZoo, primero se evalúa !raining, que es true, entonces queda sunday && true, y eso da true.

En stayingHome, como ambos son true, queda !(true && true), que es false.

Entonces el resultado final es: true- true- false.

14._B, E y G

Estas son correctas porque != funciona con objetos.

15._D

El único operador que usa tres valores es el operador ternario ?:, porque necesita una condición, un valor si es true y otro si es false. Los demás operadores solo usan uno o dos valores.

16._B

Solo hay un error en la primera línea, el 3 se convierte a long, entonces toda la operación también se vuelve long. Pero se está guardando en un int, y eso no se puede sin cast, por eso marca error.

17._C y F

Empieza en 1 y 3.

Luego en `ticketsSold += 1 + ticketsTaken++` usa el 1 de `ticketsTaken` y después lo sube a 2, entonces queda $3 + 2 = 5$.

Luego `ticketsTaken *= 2` entonces quedaría como $2 * 2 = 4$.

Y al final solo le suma 1 a `ticketsSold`, entonces queda en 6.

O sea: `ticketsTaken = 4` y `ticketsSold = 6`.

18._C

Solo los paréntesis cambian el orden de las operaciones.

Los demás símbolos no sirven para eso.

19._B y F

Empieza con `start = 7` y `end = 4`.

Luego en `end += ++start` primero aumenta `start` a 8 porque es pre-incremento, y ese 8 es el que se usa. Entonces `end` pasa de 4 a 12.

Después hacen `start = (byte)(Byte.MAX_VALUE + 1)`. Sabemos que el máximo de byte es 127, entonces $127 + 1 = 128$, pero como es byte se desborda y se vuelve -128.

Entonces al final:

`start = -128`

`end = 12`

20._A, D y E

El B y el C son incorrectos debido a que en el 'B' el menos(-) no sirve para booleanos, y el pre-incremento en el 'C' regresa el valor nuevo, no el viejo. Y en el F pues como si hay respuestas correctas queda descartada esa opción.

21._E

El `~` convierte 8 en -9, el signo negativo lo vuelve -8, no son iguales entonces toma 10, luego el `--` lo baja a 9 antes de imprimir $\rightarrow$ queda -9, -8, 9.

Review Questions

Capítulo 3

1._ A, B, C, E, F y G.

Switch si acepta enteros, char, String y enum, etc. Lo que no acepta es decimales ni long.

2._B

temperature sí cumple, entonces entra al primer if. Luego checa humidity, no es menor a 6, entonces imprime "Just Right". El último else ya no se usa.

3._A, D, F y H

El for-each funciona con arreglos y colecciones que se puedan recorrer.

Entonces arrays sí, listas y sets también, pero cosas como Object, String o Map no.

El for each dice "Te muestro todo lo que tienes en tu lista, pero yo no puedo hacer nada más que mostrartelos"

4._F

Sí se encuentra el 6 y la respuesta sería "Turtle", pero el switch está incompleto porque no tiene default ni cubre todos los casos, entonces ni siquiera compila.

5._E

El segundo for tiene un continue antes del print, entonces ese print nunca se ejecuta, y Java no permite código que nunca se va a usar, entonces no compila.

6._C, D y E

Estas son correctas debido a que el for si evalúa la condición antes de ejecutar las instrucciones dentro de él, mientras que por el lado del switch si se utiliza el String y guarda el resultado, necesitas cubrir todos los casos o poner default, y por el lado del do while si se ejecuta por lo menos una vez de entrada y luego verifica si la condición es true o false.

7._B y D

Aquí se necesita un iterador que recorra bien el arreglo, por lo tanto el B va normal en incremento, y el D va al revés, o sea en decremento, pero ambos sirven, las demás fallan por índices, sintaxis, etc.

8._G

el código no logra compilar porque se rompe debido a que el bat no existe mas haya del for en el que se declaro, podría decir que es una 'variable local' y además hay un default donde no va.

9._B, C y E

Se necesita cortar el flujo justo para que el contador llegue a 2, las opciones B, C y E con el break, continúe, lo logran. Mientras los demás o cortan muy pronto o dejan correr demasiado.

10._E

el switch está mal armado desde ahí comienzan los errores, no puedes usar un continue en los switch, también usa cosas que no son constantes y mezcla int con enum.

11._A

Aquí le está pasando el de MAMMAL, por lo tanto se va al case 3 e imprime 3.

12._C

Es correcto porque 'notes' se va acumulando dentro del while y termina en 23

13._G

El código no compila debido a que la condición del while le falta los paréntesis.

14._B, D y F

El for each agarra el tipo de cada elemento, no del contenedor, por eso: int, Character, Integer

15._F

El código no compila, porque los case están mal puestos, hay un error de sintaxis

16._A, B y D

Se necesitas recorrer el arreglo de atrás hacia adelante, esas tres sí lo hacen así, mientras que las otras o se salen del índice o se quedan en loop infinito

17._B y E

Primero el while va sumando hasta que llega a 10, entonces participants termina en 10.

Luego el do-while se ejecuta al menos una vez, por eso animals acaba en 3.

Y el for empieza en -1 y va subiendo de 2 en 2 hasta pasar el 2, entonces termina en 3.

Al final imprime 10, 3 y 3, pero como piden los distintos, son 3 y 10.

18._C y E

instanceof es la forma correcta de preguntar '¿esto es de este tipo?', por eso esa opción está bien. Y la otra dice que solo puedes usar algo cuando ya sabes qué es.

19._E

snake solo existe dentro del do, afuera ya no, entonces el código no compila.

20._A y E

Hay un ciclo infinito adentro, entonces se necesita romperlo o saltarlo. A y E sí lo evitan, las otras no.

21._E

Tiene varios errores: el tipo no sirve en el switch, hay un caso mal escrito, un punto y coma extra y un case repetido. Son 4 en total, entonces E.

22._E

Empieza en 3, el switch imprime 5. Luego el while va bajando a 2 y luego a 1, entonces imprime 2 y 1. Queda 5 2 1.

23._F

Hay un else de más que no tiene su if, entonces no compila. Por eso es F.

24._G

Está mal el for porque usa in en lugar de “:”, entonces no compila. Por eso es G.

25._D

No coincide con ningún case, entonces entra al default y como no hay break sigue sumando en los otros casos. Al final queda en 2.

26._F

El ciclo de adentro nunca termina porque r no cambia ahí, entonces se vuelve infinito.

27._F

No en todos los caminos del switch se regresa algo, por eso no compila.

28._F

Están declarando otra vez guppy donde ya existe, entonces da error y no compila.

29._C

Empieza en -2 pero como usa ++y, primero sube a -1 y ese es el primero que imprime. Va subiendo hasta 6 y ahí se detiene. Entonces queda -1 0 1 2 3 4 5 6.